

改进的生成 API （Improved Spawn API）

1. 以前的父子级生成 API

在 Bevy 中的父子级生成 API 一直以来都有些繁琐。

```
// 0      0-0      0-0-0
//              0-0-1
//          0-1
commands
    .spawn(Name::new("0"))
    .with_children(|parent| {
        parent
            .spawn(Name::new("0-0"))
            .with_children(|parent| {
                parent.spawn(Name::new("0-0-0"));
                parent.spawn(Name::new("0-0-1"));
            });
        parent.spawn(Name::new("0-1"));
    }),
```

2. 改进的父子级生成 API

Bevy 计划通过下一代场景/界面系统（BSN）大幅提升 Bevy 的生成体验。实现这一目标的重要一步是：允许开发者直接通过数据来表达层级结构，而不是使用构造器方法。关系系统的引入进一步增强了构建这种系统的价值，因为所有的关系类型都能从中受益。

在 Bevy 0.16 中，我们极大地改善了生成层级结构的易用性。现在可以这样写：

```
commands.spawn((
    Name::new("0"),
    children![
        (
            Name::new("0-0"),
            children![Name::new("0-0-0"), Name::new("0-0-1"),]
        ),
        Name::new("0-1"),
    ],
));
```

在大多数情况下，出于可读性与易用性考虑，应优先使用 children! 宏。该宏会被展开为如下等效代码：

```
commands.spawn((
    Name::new("0"),
    Children::spawn((
        Spawn((
            Name::new("0-0"),
            Children::spawn((Spawn(Name::new("0-0-0")), Spawn(Name::new("0-0-1")))),
        )),
        Spawn(Name::new("0-1")),
    )),
));
```

此外，还有一些生成封装器变体，提供了更大的灵活性：

```
commands.spawn((
    Name::new("0"),
    Children::spawn((
        // 用闭包的方式生成子实体
        SpawnWith(|parent: &mut ChildSpawner| {
            parent.spawn((
                Name::new("0-0"),
                Children::spawn(
                    // 用迭代器的方式生成子实体
                    SpawnIter(["0-0-0", "0-0-1"].into_iter().map(Name::new)),
                ),
            ));
            parent.spawn(Name::new("0-1"));
        })),
    )),
));
```

3. 关系生成 API

值得注意的是，这套 API 支持所有类型的关系。例如，你可以生成一个 Likes / LikedBy 关系层级（参考前面关系系统部分的定义）：

```
commands.spawn((
    Name::new("0"),
    related!(LikedBy [
        (
            Name::new("0-0"),
            related!(LikedBy [
                Name::new("0-0-0"),
                Name::new("0-0-1"),
            ]),
        ),
        Name::new("0-1"),
    ]),
));
```

在大多数情况下，出于可读性与易用性考虑，应优先使用 related! 宏。该宏会被展开为如下等效代码：

```
commands.spawn((
    Name::new("0"),
    LikedBy::spawn((
        Spawn((
            Name::new("0-0"),
            LikedBy::spawn((Spawn(Name::new("0-0-0")), Spawn(Name::new("0-0-1")))),
        )),
        Spawn(Name::new("0-1")),
    )),
));
```

此外，还有一些生成封装器变体，提供了更大的灵活性：

```
commands.spawn((
    Name::new("0"),
    LikedBy::spawn(SpawnWith(|parent: &mut RelatedSpawner<Likes>| {
        parent.spawn((
            Name::new("0-0"),
            LikedBy::spawn(SpawnIter(["0-0-0", "0-0-1"].into_iter().map(Name::new))),
        ));
        parent.spawn(Name::new("0-1"));
    })),
));
```

4. 性能

这套 API 还使得我们能优化层级构建的性能，通过减少内存重新分配（re-allocations）。因为在多数情况下（例如不使用 SpawnWith 的情况），我们可以静态地确定一个实体会拥有多少相关实体，并为关系目标组件（如 Children）预先分配空间。